SLIIT
Discover Your Future

SE2020 – Web Technologies and Mobile

Lecture 12

NoSQL and MongoDB

SLIIT
FACULTY OF COMPUTING

Agenda

- ❑ Fundamental of NoSQL
- ❑ CAP Theorem
- ❑ Different types of NoSQL DB
- ❑ Introduction to MongoDB
- ❑ Integrate MongoDB with Express

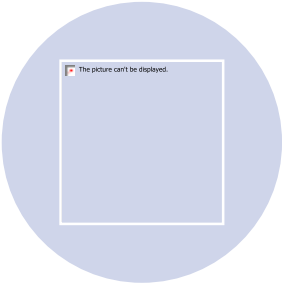
Fundamental of NoSQL

- NoSQL (Not Only SQL) is a modern database approach that differs from traditional relational database systems.
- It is designed to store, retrieve, and manage **large volumes of unstructured, semi-structured, or rapidly changing data** efficiently.
- Unlike relational databases that rely on fixed schemas and tables, NoSQL databases offer flexibility, scalability, and high performance for modern, data-intensive applications such as social networks, IoT systems, and real-time analytics.

NoSQL vs Relational DB

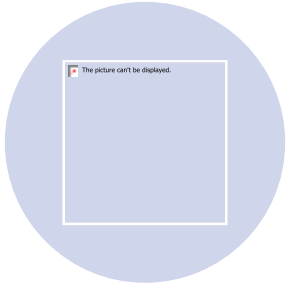
	Relational Databases	Non-Relational Databases
Structure	Structured, with defined relationships between tables	Flexible, allowing for dynamic and changing data structures
Query Language	Uses SQL for querying data	Uses various querying languages, including JSON and BSON
Data Consistency	Enforces strict data consistency rules	Often prioritizes availability and partition tolerance over consistency
Popularity	Widely used and has a long history in the tech industry	Gained popularity with the rise of big data and modern web applications
Transactions and Operations	Well-suited for complex transactions and operations	Often faster and more scalable than relational databases
Purpose	Best suited for structured data and complex queries	Best suited for unstructured data and high scalability
Scalability	Vertical scaling is more common, but limited	Horizontal scaling is more common, and highly scalable
Examples	MySQL, Oracle, Microsoft SQL Server	MongoDB, Cassandra, Couchbase
Data Volume	Good for small to medium data volumes	Good for large data volumes
Cost	Relatively expensive	Relatively cheap
Use Cases	Financial institutions, transactional systems, inventory management	Big data, IoT, content management, cloud computing

NoSQL vs Relational DB – Real world example



In a relational database, a book record is often dissembled (or "normalized") and stored in separate tables, and relationships are defined by primary and foreign key constraints. In this example, the ***Books table*** has columns for *ISBN, Book Title, and Edition Number*, the ***Authors table*** has columns for *AuthorID and Author Name*, and finally, the ***Author-ISBN table*** has columns for *AuthorID and ISBN*. The relational model is designed to enable the database to enforce referential integrity between tables in the database, normalized to reduce the redundancy, and generally optimized for storage.

In a NoSQL database, a **book record** is usually stored as a JSON document. For each book, the *item, ISBN, Book Title, Edition Number, Author Name, and AuthorID* are stored as attributes in a single document. In this model, data is optimized for intuitive development and horizontal scalability.



NoSQL – Key Characteristics

- **Schema-less Structure** – Data can be stored without predefined tables or columns, allowing flexibility.
- **Horizontal Scalability** – Supports easy expansion by adding more servers instead of upgrading existing hardware.
- **High Performance** – Optimized for fast read/write operations and low latency.
- **Flexible Data Models** – Supports multiple data types such as document, key-value, column, and graph.
- **Distributed and Fault-Tolerant** – Built to operate efficiently across multiple nodes with automatic replication and recovery.

Types of NoSQL

- Document Based Databases
- Key-value Stores
- Graph Databases
- Column Family Stores
- In-memory

Document Based Databases

- Document databases store data in structured documents usually in **JSON, BSON, or XML** format, rather than rows and columns.
- Each document contains key-value pairs and can have a flexible structure, allowing different documents in the same collection to have different fields.
- Schema-less design – no need for a fixed table schema.
- Hierarchical data storage format supports nested structures.
- Easy mapping between application objects and database records.
- Example: **MongoDB, CouchDB, Firebase FireStore**

```
{
  "product_id": "P001",
  "name": "Wireless Mouse",
  "price": 1200,
  "features": ["Bluetooth", "Rechargeable", "Ergonomic"],
  "stock": 40
}
```

Key Value Stores

- Key-Value databases are the simplest type of NoSQL databases.
- Data is stored as a collection of key-value pairs, where a unique key is used to retrieve a corresponding value.
- The value can be a string, JSON object, or binary data.
- Extremely fast read/write operations.
- Ideal for caching and session management.
- Highly scalable and distributed by design.
- Simple data model, making it easy to implement.
- Example: **Redis, Amazon DynamoDB, Riak**

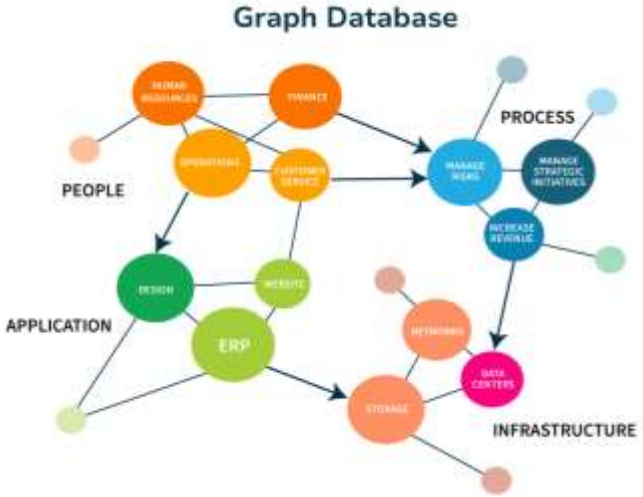
Key	Value
"user123"	{"name": “Sanduni", "email": “sanduni@gmail.com"}
"session456"	{"token": "abc123xyz", "expires": "2025-10-29T10:00:00Z"}

Column Family Databases

- Column-family databases (also known as wide-column stores) organize data into columns and column families rather than rows and tables.
- Each column family contains rows identified by a key, but each row can have a different set of columns.
- This makes them efficient for analytical queries and handling large, sparse datasets.
- Optimized for high-performance read/write of large datasets.
- **Example:** Apache Cassandra, HBase, ScyllaDB

Graph Databases

- Graph databases store data in nodes (entities) and edges (relationships), making them ideal for representing highly connected data.
- Each node and edge can have attributes that describe entities and their relationships.
- Graph databases use query languages such as Cypher (Neo4j) or Gremlin to explore relationships efficiently.
- Designed for relationship-heavy data and intuitive visualization of data relationships.
- **Example:** Neo4j, Amazon Neptune, ArangoDB



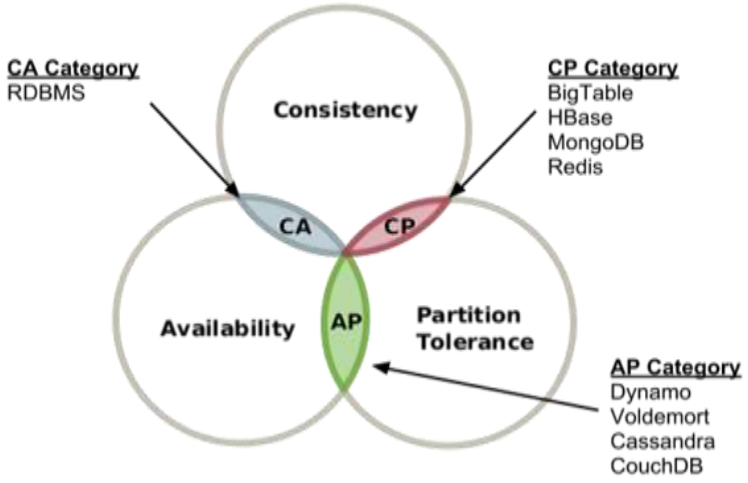
Exercise:

Discuss the advantages and limitations of NoSQL databases?

CAP Theorem

It explains the trade-offs in distributed databases. It states that a distributed system can only guarantee two out of the three properties

- **Consistency** - Every read receives the most recent write (all nodes see the same data at the same time).
- **Availability** - Every request receives a response (even if some nodes are down).
- **Partition Tolerance** - The system continues to function even if communication between some nodes fails or is delayed (network partition).



MongoDB

- MongoDB is a popular NoSQL, document-oriented database system designed to store and manage large volumes of data in a flexible, scalable, and high-performance manner.
- It uses JSON-like documents (BSON – Binary JSON) to store data, making it ideal for modern applications that handle unstructured or semi-structured information.
- MongoDB was developed by MongoDB Inc. and first released in 2009 and has become one of the most widely used NoSQL databases in the world.

MongoDB – Key Characteristics

- **Document-Oriented** Storage. Data is stored in documents (similar to JSON objects) instead of rows and columns. Each document represents a record and contains key-value pairs.
- **Schema-Less** Design. Documents in the same collection can have different structures. Ideal for rapidly evolving applications.
- High **Scalability**. Supports horizontal scaling through sharding, distributing data across multiple servers.
- High **Availability**. Achieved through replication (Replica Sets) – multiple copies of data ensure fault tolerance.

MongoDB – Data Structure

Level	Description	Example
Database	Highest level container for collections.	UniversityDB
Collection	Group of related documents (like a table in RDBMS)	Students
Document	Individual record stored in JSON/ BSON format	{“name” : “Sanduni”, “age” : 27}
Field	Key-value pair inside a document	“name” : “Sanduni”

MongoDB – Common Operations

Operation	Description	Example (MQL)
Insert	Add a new document	db.students.insertOne({name: “Sanduni", age: 25})
Find	Retrieve documents	db.students.find({city: "Wattala"})
Update	Modify existing document	db.students.updateOne({name: “Sanduni"}, {$set: {age: 26}})
Delete	Remove a document	db.students.deleteOne({name: “Sanduni"})

Tutorial– Connecting MongoDB with Backend Server

SLIIT
Discover Your Future

Thank You